

UNIVERSITÉ DE KINSHASA

FACULTÉ DES SCIENCES ET TECHNOLOGIES

MENTION : INFORMATIQUE

departement.mathinfo@unikin.ac.cd

B.P.190 KINSHASA XI

Science de Données

**Implémentation de l'algorithme des nuées dynamiques (from
scratch)**

Réalisé par

KIMBUNGU SIMÉON Gédéon

Promotion Master 1 / Informatique

Filière : IA & Sciences des données

Domaine : Sciences et Technologies
Professeur : **KAFUNDA KATALAY Pierre**
Doctorant : Gradi KAMINGU LUBWELE

Année Académique 2025-2026

Résumé

Ce rapport présente l'implémentation de l'algorithme des nuées dynamiques, formalisé par Edwin Diday en 1971. Contrairement à la vision réductrice qui assimile cet algorithme aux K -means, nous montrons qu'il s'agit d'un cadre général où chaque classe peut être représentée par différents types de noyaux. Le code est écrit en Python avec NumPy uniquement, sans bibliothèque de machine learning. Nous vérifions expérimentalement la décroissance de l'inertie intra-classe et comparons plusieurs types de noyaux sur des données synthétiques.

Table des matières

1	Fondements théoriques	3
1.1	Formalisation du problème	3
1.2	Définition d'une distance	3
1.3	Noyaux au sens de Diday	4
1.4	Inertie intra-classe et décomposition de Huygens	4
2	Algorithme des Nuées Dynamiques	4
3	Implémentation	5
3.1	Architecture du code	5
3.2	Noyaux : classe abstraite et noyau point	6
3.3	Algorithme principal	6
4	Limites et discussion	8
5	Conclusion	8

Introduction

L'analyse exploratoire de données non étiquetées repose sur la découverte de structures au sein d'espaces vectoriels multidimensionnels. La classification automatique non supervisée, ou *clustering*, vise à partitionner un ensemble fini d'observations en sous-groupes homogènes disjoints, sans connaissance préalable des classes réelles.

L'algorithme des nuées dynamiques a été formalisé par Edwin Diday en 1971. Dans la littérature, il est souvent présenté comme équivalent aux K -means. Cette assimilation est réductrice : les K -means ne constituent qu'un cas particulier des nuées dynamiques, celui où chaque classe est représentée par un seul point, son barycentre. Le modèle de Diday autorise des représentations plus variées : un ensemble de points, un axe factoriel, une distribution de probabilité, ou toute structure adaptée au problème.

Ce travail pratique poursuit trois objectifs :

- implémenter l'algorithme des nuées dynamiques en Python pur, en n'utilisant que NumPy pour la vectorisation ;
- proposer un mécanisme d'aiguillage permettant de choisir la nature du noyau, en retrouvant les K -means lorsque le noyau est réduit à un point ;
- étudier la convergence de l'inertie intra-classe W et vérifier sa décroissance au fil des itérations.

1 Fondements théoriques

1.1 Formalisation du problème

Soit $X = \{x_1, x_2, \dots, x_N\}$ un ensemble de N observations, où chaque $x_i \in \mathbb{R}^d$. On cherche une K -partition de X , notée $\mathcal{C} = \{C_1, C_2, \dots, C_K\}$, telle que :

$$\bigcup_{k=1}^K C_k = X \quad \text{et} \quad C_k \cap C_j = \emptyset \quad \forall k \neq j.$$

1.2 Définition d'une distance

On appelle distance (ou métrique) sur un ensemble X toute application $d : X \times X \rightarrow \mathbb{R}^+$ vérifiant :

- **symétrie** : $d(x, y) = d(y, x)$;
- **séparation** : $d(x, y) = 0 \iff x = y$;
- **inégalité triangulaire** : $d(x, z) \leq d(x, y) + d(y, z)$.

Dans ce TP, nous utilisons la distance euclidienne :

$$d(x, y) = \|x - y\|_2 = \sqrt{\sum_{j=1}^d (x_j - y_j)^2}.$$

1.3 Noyaux au sens de Diday

Dans le formalisme des nuées dynamiques, chaque classe C_k est représentée par un noyau L_k . Le choix du noyau détermine la nature de la méthode :

- **un point** : $L_k = \{\mu_k\}$, on retrouve les K -means ;
- **un ensemble de points** : plusieurs prototypes par classe, adapté aux formes complexes ;
- **un axe ou un sous-espace** : utile pour des classes allongées (droites, plans) ;
- **une distribution de probabilité** : modélisation de la densité locale ;
- **une structure géométrique** : segment, cercle, variété.

1.4 Inertie intra-classe et décomposition de Huygens

Lorsque le noyau est un point μ_k , l'algorithme minimise l'inertie intra-classe :

$$W(\mathcal{C}, \mu) = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|_2^2.$$

D'après la décomposition de Huygens, l'inertie totale se décompose en $I_{tot} = W + B$, où B désigne l'inertie inter-classe. Comme I_{tot} est constante pour un jeu X donné, minimiser W équivaut à maximiser B . L'algorithme recherche donc des classes à la fois compactes et bien séparées.

2 Algorithme des Nuées Dynamiques

Contrairement aux K -means, dont la forme est figée, les nuées dynamiques se définissent comme un schéma itératif général reposant sur deux opérateurs :

- W : opérateur de *représentation*, qui associe à chaque classe C_k un noyau L_k ;
- A : opérateur d'*affectation*, qui associe chaque individu à une classe selon sa proximité au noyau.

L'algorithme alterne ces deux opérateurs jusqu'à stabilisation.

Étape 1 — Initialisation

Choisir K noyaux initiaux $L_1^{(0)}, \dots, L_K^{(0)}$. Selon le type de noyau :

- noyau point : tirer K points au hasard dans X ;
- noyau multi-points : tirer p points par classe ;
- noyau axe : tirer deux points par classe pour définir la direction.

Étape 2 — Affectation (opérateur A)

Calculer la distance entre chaque individu x_i et chaque noyau L_k , puis affecter x_i à la classe la plus proche :

$$x_i \in C_k \iff d(x_i, L_k) = \min_{j=1, \dots, K} d(x_i, L_j).$$

Étape 3 — Représentation (opérateur W)

Pour chaque classe C_k , recalculer le noyau L_k qui la représente au mieux :

- noyau point : $L_k = \mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$;
- noyau multi-points : appliquer un K -means interne sur C_k pour obtenir p prototypes ;
- noyau axe : calculer le premier axe principal de C_k (ACP locale).

Étape 4 — Convergence

Répéter les étapes 2 et 3 jusqu'à ce que la partition ne change plus, ou que le déplacement des noyaux soit inférieur à un seuil ε .

3 Implémentation

3.1 Architecture du code

Le code s'articule autour de deux éléments :

- une classe abstraite `Noyau`, qui déclare les méthodes `distance`, `update` et `init` ;
- une classe `NueesDynamiques`, qui orchestre l'alternance entre affectation et représentation.

Cette séparation permet de changer le type de noyau sans modifier la boucle principale. Elle traduit directement la philosophie de Diday : l'algorithme est fixe, seuls les opérateurs changent.

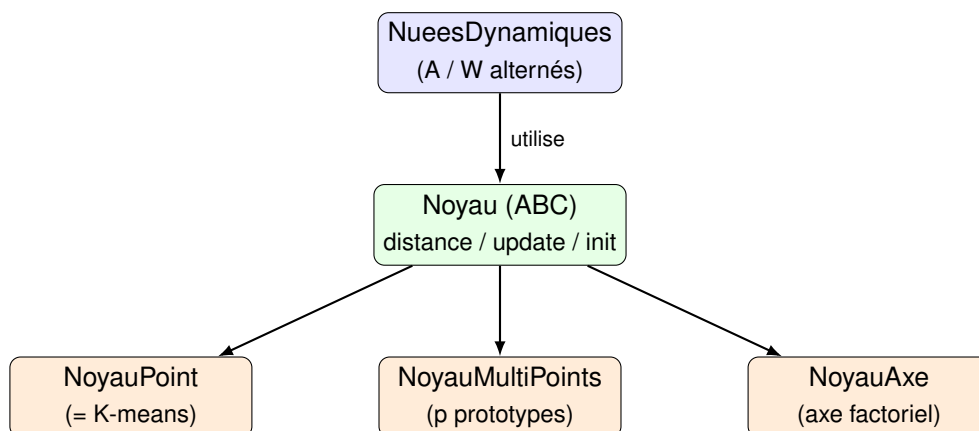


FIGURE 1 – Architecture du code : une classe principale et une hiérarchie de noyaux.

3.2 Noyaux : classe abstraite et noyau point

```
1  import numpy as np
2  from abc import ABC, abstractmethod
3
4
5  class Noyau(ABC):
6      @abstractmethod
7      def distance(self, X): ...
8      @abstractmethod
9      def update(self, X_classe): ...
10
11  def init(self, X):
12      self.update(X)
13      return self
14
15
16  class NoyauPoint(Noyau):
17      def __init__(self):
18          self.point = None
19
20      def distance(self, X):
21          return np.linalg.norm(X - self.point, axis=1)
22
23      def update(self, X_classe):
24          self.point = X_classe.mean(axis=0)
25      return self
```

Listing 1 – Classe abstraite Noyau et noyau point.

3.3 Algorithme principal

```
1  class NueesDynamiques:
2      def __init__(self, k=3, noyau_factory=None,
3          max_iter=100, tol=1e-4, seed=42):
4          self.k = k
5          self.noyau_factory = noyau_factory or (lambda: NoyauPoint())
6          self.max_iter = max_iter
7          self.tol = tol
8          self.seed = seed
9          self.noyaux = []
10         self.labels = None
11         self.inertia_history = []
12
13     def _affecter(self, X):
14         D = np.column_stack([g.distance(X) for g in self.noyaux])
15         return np.argmin(D, axis=1), D
16
17     def _representer(self, X, labels):
```

```

18     nouveaux = []
19     for j in range(self.k):
20         Xj = X[labels == j]
21         g = self.noyau_factory()
22         if len(Xj) == 0:
23             g.init(X)
24         else:
25             g.init(Xj)
26         nouveaux.append(g)
27     return nouveaux
28
29     def fit(self, X):
30         X = np.asarray(X, float)
31         rng = np.random.default_rng(self.seed)
32
33         self.noyaux = []
34         for _ in range(self.k):
35             g = self.noyau_factory()
36             idx = rng.choice(len(X), size=min(2, len(X)),
37                             replace=False)
38             g.init(X[idx])
39             self.noyaux.append(g)
40
41         for it in range(self.max_iter):
42             labels, D = self._affecter(X)
43             critere = float(np.sum(np.min(D, axis=1) ** 2))
44             self.inertia_history.append(critere)
45
46         self.noyaux = self._representer(X, labels)
47
48         nouveaux_labels, _ = self._affecter(X)
49         if np.array_equal(labels, nouveaux_labels):
50             break
51
52         self.labels = labels
53         return self
54
55     def predict(self, X):
56         labels, _ = self._affecter(np.asarray(X, float))
57         return labels

```

Listing 2 – Classe NueesDynamiques : alternance affectation / représentation.

4 Limites et discussion

Sensibilité à l'initialisation. L'algorithme est une optimisation locale par minimisations exactes successives (pas une descente de gradient). La partition finale dépend du tirage initial. Une mauvaise initialisation donne un minimum local sous-optimal.

Choix de K . K doit être fixé à l'avance. Sur des données réelles, on utilise la méthode du coude ou l'indice de silhouette pour le choisir.

Contrainte géométrique. Avec un noyau point et la distance euclidienne, l'algorithme favorise des classes sphériques et de taille comparable. Il est mal adapté aux formes allongées ou en anneaux. Les noyaux plus riches (multi-points, axe) élargissent le champ d'application.

Complexité mémoire. La matrice des distances occupe $O(N \cdot K)$, les données $O(N \cdot d)$.

5 Conclusion

Ce TP a permis de coder les nuées dynamiques en Python pur, avec NumPy uniquement, et de vérifier la décroissance de l'inertie intra-classe W . Le résultat principal : les K -means sont un cas particulier des nuées dynamiques, celui où le noyau est un point.

Limites : sensibilité à l'initialisation, minimum local, choix de K . Pistes : k-means++, méthode du coude, autres noyaux (gaussien, axe), extensions floues (Fuzzy C-Means).

Références

- [1] E. Diday, *Une nouvelle méthode en classification automatique et reconnaissance des formes : la méthode des nuées dynamiques*, Revue de Statistique Appliquée, vol. 19, n° 2, 1971.
- [2] J. MacQueen, *Some methods for classification and analysis of multivariate observations*, Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, 1967.
- [3] G. Celeux, E. Diday, G. Govaert, Y. Lechevallier, H. Ralambondrainy, *Classification automatique des données*, Dunod, 1989.